

Predicting Hospital Readmission using Python and SQL

By Jack Guo

Introduction

Hospital readmission rates are often utilized as a measure of care quality and patient outcomes. The Center for Medicare and Medicaid services (CMS) has penalized hospitals for excess readmission rates since 2012 under the Hospital Readmissions Reduction Program ¹. Identifying patients at high risk of readmission at the time of discharge allows clinicians to potentially reduce patient burden and hospital costs.

This project uses the MIMIC-IV Clinical Database Demo, a publicly available de-identified dataset containing clinical records from patients admitted to Beth Israel Deaconess Medical Center in Boston, Massachusetts. This demo dataset contains records for approximately 100 patients across multiple relational tables covering hospital admissions, ICU stays, laboratory results, diagnoses, and more.

The primary goal of this project is to build a logistic regression model predicting whether a patient will be readmitted in 30 days using features derived from demographics, admission characteristics, and ICU utilization. A second goal is to demonstrate a reproducible clinical data pipeline: raw relational data extracted using SQL, processed in Python, and analyzed using standard regression methods.

The analysis proceeds as follows: exploratory data analysis characterizes the patient cohort and admission patterns; feature engineering constructs the target variable and predictors from multiple joined tables; a logistic regression model is then fit and evaluated using area under the ROC curve (AUC) as the primary performance metric.

Exploratory Data Analysis

```
In [2]: import os
import pandas as pd
import sqlite3
import matplotlib.pyplot as plt

os.chdir("C:/Users/flami/Documents/MIMIC-IV Analysis/MIMIC-IV-EDA")

# create sqlite db
root_folder = "."
conn = sqlite3.connect("mimic.db")
```

```

for root, dirs, files in os.walk(root_folder):
    for filename in files:
        if filename.endswith(".csv"):
            filepath = os.path.join(root, filename)

            # change file names to folder_name
            if root_folder == root:
                tablename = filename.replace(".csv", "")
            else:
                subfolder = os.path.basename(root)
                tablename = f"{subfolder}_{filename}".replace(".csv", "")

            # read to db
            df = pd.read_csv(filepath)
            df.to_sql(tablename, conn, if_exists="replace", index=False)

```

C:\Users\flami\AppData\Local\Temp\ipykernel_9396\2723046639.py:25: DtypeWarning: Columns (0: administration_type, 1: barcode_type, 2: reason_for_no_barcode, 3: complete_dose_not_given, 4: dose_due, 5: dose_due_unit, 6: dose_given, 7: dose_given_unit, 8: will_remainder_of_dose_be_given, 9: product_unit, 10: product_code, 11: product_description, 12: product_description_other, 13: infusion_rate_adjustment, 14: infusion_rate_unit, 15: route, 16: infusion_complete, 17: completion_interval, 18: new_iv_bag_hung, 19: continued_infusion_in_other_location, 20: restart_interval, 21: side, 22: site, 23: non_formulary_visual_verification) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv(filepath)
```

Here, the tables are previewed, with the schema and top entries shown for the important tables.

In [3]: *# preview tables*

```

tables = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name")
display(tables.head(5))

```

	name
0	demo_subject_id
1	dur_by_type
2	hosp_admissions
3	hosp_d_hcpcs
4	hosp_d_icd_diagnoses

In [4]: *# get schema and preview a table*

```

for table in ("hosp_patients", "hosp_admissions", "hosp_labevents", "icu_icustays"):

    schema = pd.read_sql(f"PRAGMA table_info({table})", conn)
    print(f"Schema for {table}:")
    print(schema)

```

```
preview = pd.read_sql(f"SELECT * FROM {table} LIMIT 5", conn)
print(f"Preview for {table}:")
print(preview)
print("\n")
```

Schema for hosp_patients:

	cid	name	type	notnull	dflt_value	pk
0	0	subject_id	INTEGER	0	None	0
1	1	gender	TEXT	0	None	0
2	2	anchor_age	INTEGER	0	None	0
3	3	anchor_year	INTEGER	0	None	0
4	4	anchor_year_group	TEXT	0	None	0
5	5	dod	TEXT	0	None	0

Preview for hosp_patients:

	subject_id	gender	anchor_age	anchor_year	anchor_year_group	dod
0	10014729	F	21	2125	2011 - 2013	NaN
1	10003400	F	72	2134	2011 - 2013	2137-09-02
2	10002428	F	80	2155	2011 - 2013	NaN
3	10032725	F	38	2143	2011 - 2013	2143-03-30
4	10027445	F	48	2142	2011 - 2013	2146-02-09

Schema for hosp_admissions:

	cid	name	type	notnull	dflt_value	pk
0	0	subject_id	INTEGER	0	None	0
1	1	hadm_id	INTEGER	0	None	0
2	2	admittime	TEXT	0	None	0
3	3	dischtime	TEXT	0	None	0
4	4	deathtime	TEXT	0	None	0
5	5	admission_type	TEXT	0	None	0
6	6	admit_provider_id	TEXT	0	None	0
7	7	admission_location	TEXT	0	None	0
8	8	discharge_location	TEXT	0	None	0
9	9	insurance	TEXT	0	None	0
10	10	language	TEXT	0	None	0
11	11	marital_status	TEXT	0	None	0
12	12	race	TEXT	0	None	0
13	13	edregtime	TEXT	0	None	0
14	14	edouttime	TEXT	0	None	0
15	15	hospital_expire_flag	INTEGER	0	None	0

Preview for hosp_admissions:

	subject_id	hadm_id	admittime	dischtime	\
0	10004235	24181354	2196-02-24 14:38:00	2196-03-04 14:02:00	
1	10009628	25926192	2153-09-17 17:08:00	2153-09-25 13:20:00	
2	10018081	23983182	2134-08-18 02:02:00	2134-08-23 19:35:00	
3	10006053	22942076	2111-11-13 23:39:00	2111-11-15 17:20:00	
4	10031404	21606243	2113-08-04 18:46:00	2113-08-06 20:57:00	

	deathtime	admission_type	admit_provider_id	\
0	NaN	URGENT	P03YMR	
1	NaN	URGENT	P41R5N	
2	NaN	URGENT	P233F6	
3	2111-11-15 17:20:00	URGENT	P38TI6	
4	NaN	URGENT	P07HDB	

	admission_location	discharge_location	insurance	language	\
0	TRANSFER FROM HOSPITAL	SKILLED NURSING FACILITY	Medicaid	ENGLISH	
1	TRANSFER FROM HOSPITAL	HOME HEALTH CARE	Medicaid	?	
2	TRANSFER FROM HOSPITAL	SKILLED NURSING FACILITY	Medicare	ENGLISH	
3	TRANSFER FROM HOSPITAL	DIED	Medicaid	ENGLISH	
4	TRANSFER FROM HOSPITAL	HOME	Other	ENGLISH	

	marital_status	race	edregtime \
0	SINGLE	BLACK/CAPE VERDEAN	2196-02-24 12:15:00
1	MARRIED	HISPANIC/LATINO - PUERTO RICAN	NaN
2	MARRIED	WHITE	2134-08-17 16:24:00
3	NaN	UNKNOWN	NaN
4	WIDOWED	WHITE	NaN

	edouttime	hospital_expire_flag
0	2196-02-24 17:07:00	0
1	NaN	0
2	2134-08-18 03:15:00	0
3	NaN	1
4	NaN	0

Schema for hosp_labevents:

	cid	name	type	notnull	dflt_value	pk
0	0	labevent_id	INTEGER	0	None	0
1	1	subject_id	INTEGER	0	None	0
2	2	hadm_id	REAL	0	None	0
3	3	specimen_id	INTEGER	0	None	0
4	4	itemid	INTEGER	0	None	0
5	5	order_provider_id	TEXT	0	None	0
6	6	charttime	TEXT	0	None	0
7	7	storetime	TEXT	0	None	0
8	8	value	TEXT	0	None	0
9	9	valuenum	REAL	0	None	0
10	10	valueuom	TEXT	0	None	0
11	11	ref_range_lower	REAL	0	None	0
12	12	ref_range_upper	REAL	0	None	0
13	13	flag	TEXT	0	None	0
14	14	priority	TEXT	0	None	0
15	15	comments	TEXT	0	None	0

Preview for hosp_labevents:

	labevent_id	subject_id	hadm_id	specimen_id	itemid	order_provider_id \
0	172061	10014354	29600294.0	1808066	51277	None
1	172062	10014354	29600294.0	1808066	51279	None
2	172068	10014354	29600294.0	1808066	52172	None
3	172063	10014354	29600294.0	1808066	51301	None
4	172050	10014354	29600294.0	1808066	51249	None

	charttime	storetime	value	valuenum	valueuom \
0	2148-08-16 00:00:00	2148-08-16 01:30:00	15.4	15.40	%
1	2148-08-16 00:00:00	2148-08-16 01:30:00	3.35	3.35	m/uL
2	2148-08-16 00:00:00	2148-08-16 01:30:00	49.7	49.70	fL
3	2148-08-16 00:00:00	2148-08-16 01:30:00	20.3	20.30	K/uL
4	2148-08-16 00:00:00	2148-08-16 01:30:00	31.1	31.10	g/dL

	ref_range_lower	ref_range_upper	flag	priority	comments
0	10.5	15.5	NaN	ROUTINE	None
1	4.6	6.1	abnormal	ROUTINE	None
2	35.1	46.3	abnormal	ROUTINE	None
3	4.0	10.0	abnormal	ROUTINE	None
4	32.0	37.0	abnormal	ROUTINE	None

Schema for icu_icustays:

	cid	name	type	notnull	dflt_value	pk
0	0	subject_id	INTEGER	0	None	0
1	1	hadm_id	INTEGER	0	None	0
2	2	stay_id	INTEGER	0	None	0
3	3	first_careunit	TEXT	0	None	0
4	4	last_careunit	TEXT	0	None	0
5	5	intime	TEXT	0	None	0
6	6	outtime	TEXT	0	None	0
7	7	los	REAL	0	None	0

Preview for icu_icustays:

	subject_id	hadm_id	stay_id	\
0	10018328	23786647	31269608	
1	10020187	24104168	37509585	
2	10020187	26842957	32554129	
3	10012853	27882036	31338022	
4	10020740	25826145	32145159	

	first_careunit	last_careunit	\
0	Neuro Stepdown	Neuro Stepdown	
1	Neuro Surgical Intensive Care Unit (Neuro SICU)	Neuro Stepdown	
2	Neuro Intermediate	Neuro Intermediate	
3	Trauma SICU (TSICU)	Trauma SICU (TSICU)	
4	Trauma SICU (TSICU)	Trauma SICU (TSICU)	

	intime	outtime	los
0	2154-04-24 23:03:44	2154-05-02 15:55:21	7.702512
1	2169-01-15 04:56:00	2169-01-20 15:47:50	5.452662
2	2170-02-24 18:18:46	2170-02-25 15:15:26	0.872685
3	2176-11-26 02:34:49	2176-11-29 20:58:54	3.766725
4	2150-06-03 20:12:32	2150-06-04 21:05:58	1.037106

We check for missing and distinct values in the set of important tables. While there are missing values for some of the variables in a few tables, no variable with missing values will be used in our regression model so we choose not to remedy them. Having 100 distinct values for each table means that we have 100 unique patient entries among the raw data.

```
In [5]: for table in ("hosp_patients", "hosp_admissions", "hosp_labevents", "icu_icustays")
df = pd.read_sql(f"SELECT * FROM {table}", conn)
print(f"{table} Missing Values:")
print(df.isnull().sum())
```

```
hosp_patients Missing Values:
subject_id      0
gender          0
anchor_age      0
anchor_year     0
anchor_year_group 0
dod             69
dtype: int64
hosp_admissions Missing Values:
subject_id      0
hadm_id         0
admittime       0
disctime        0
deathtime      260
admission_type  0
admit_provider_id 0
admission_location 0
discharge_location 42
insurance       0
language        0
marital_status  12
race            0
edregtime       93
edouttime       93
hospital_expire_flag 0
dtype: int64
hosp_labevents Missing Values:
labevent_id     0
subject_id      0
hadm_id         28420
specimen_id     0
itemid          0
order_provider_id 90897
charttime       0
storetime       992
value           9588
valuenum        12481
valueuom        16203
ref_range_lower 18728
ref_range_upper 18728
flag            67452
priority        9329
comments        89273
dtype: int64
icu_icustays Missing Values:
subject_id      0
hadm_id         0
stay_id         0
first_careunit  0
last_careunit   0
intime          0
outtime         0
los             0
dtype: int64
```

```
In [6]: for table in ("hosp_patients", "hosp_admissions", "hosp_labevents", "icu_icustays")
```

```
df = pd.read_sql(f"SELECT COUNT(DISTINCT subject_id) AS dist FROM {table}", con
print(f"{table} DISTINCT Values:")
print(df)
```

hosp_patients DISTINCT Values:

```
dist
```

```
0    100
```

hosp_admissions DISTINCT Values:

```
dist
```

```
0    100
```

hosp_labevents DISTINCT Values:

```
dist
```

```
0    100
```

icu_icustays DISTINCT Values:

```
dist
```

```
0    100
```

MIMIC-IV de-identifies patients, meaning rather than an exact age, we're given an anchor age. An anchor age is the age of a patient taken within a three year window. While this introduces uncertainty if we use age as a predictor, older anchor ages still refer to older patients and are associated with a higher baseline risk.

We can see that most patients lie within the 45-90 age range, with the 65-70 age group being larger and below 45 groups being smaller.

Most of patient stays last around 6 days or less, with a sharp decline in the number of patients experiencing stays of higher duration.

```
In [7]: # age distribution for hosp patients
hosp_pat_ages = pd.read_sql("""
SELECT anchor_age, COUNT(*) as n FROM hosp_patients
GROUP BY anchor_age
ORDER BY anchor_age
""", conn)

plt.hist(hosp_pat_ages["anchor_age"], weights=hosp_pat_ages["n"])
plt.title("Hospital Patient Anchor Age Distribution")
plt.xlabel("Anchor Age")
plt.ylabel("Count")
plt.show()

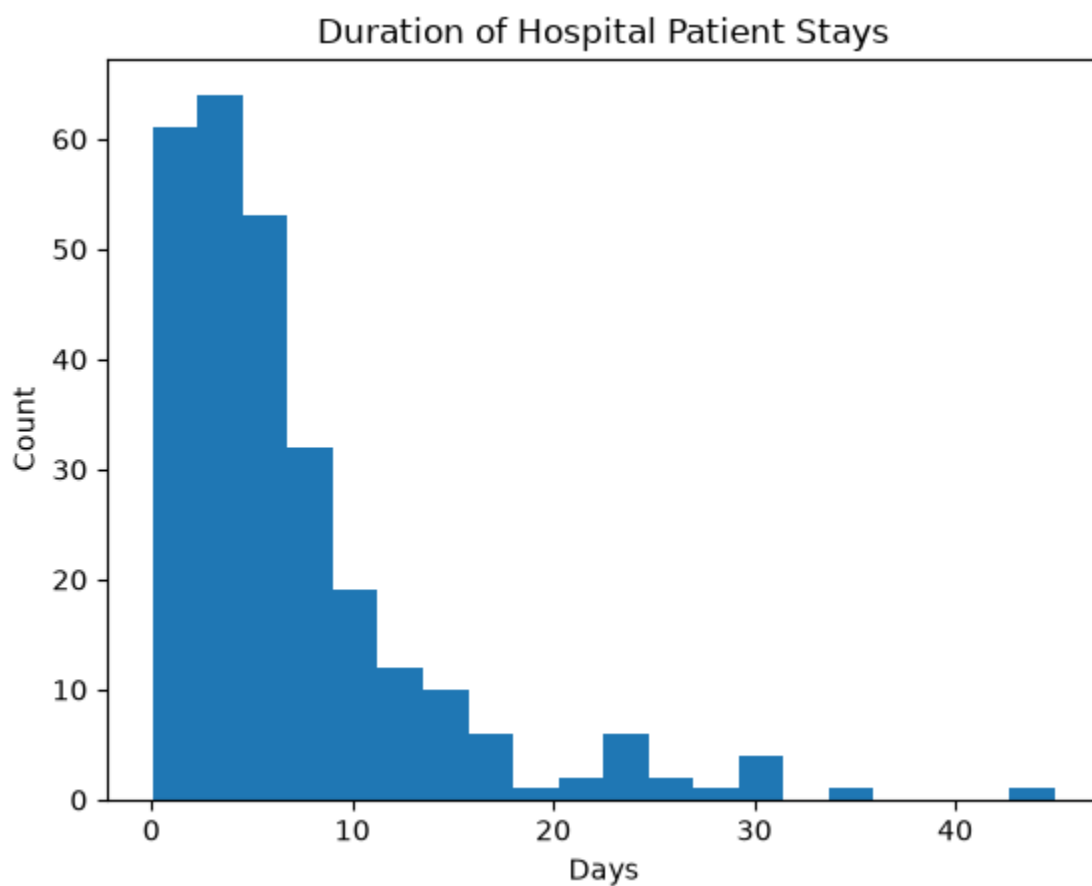
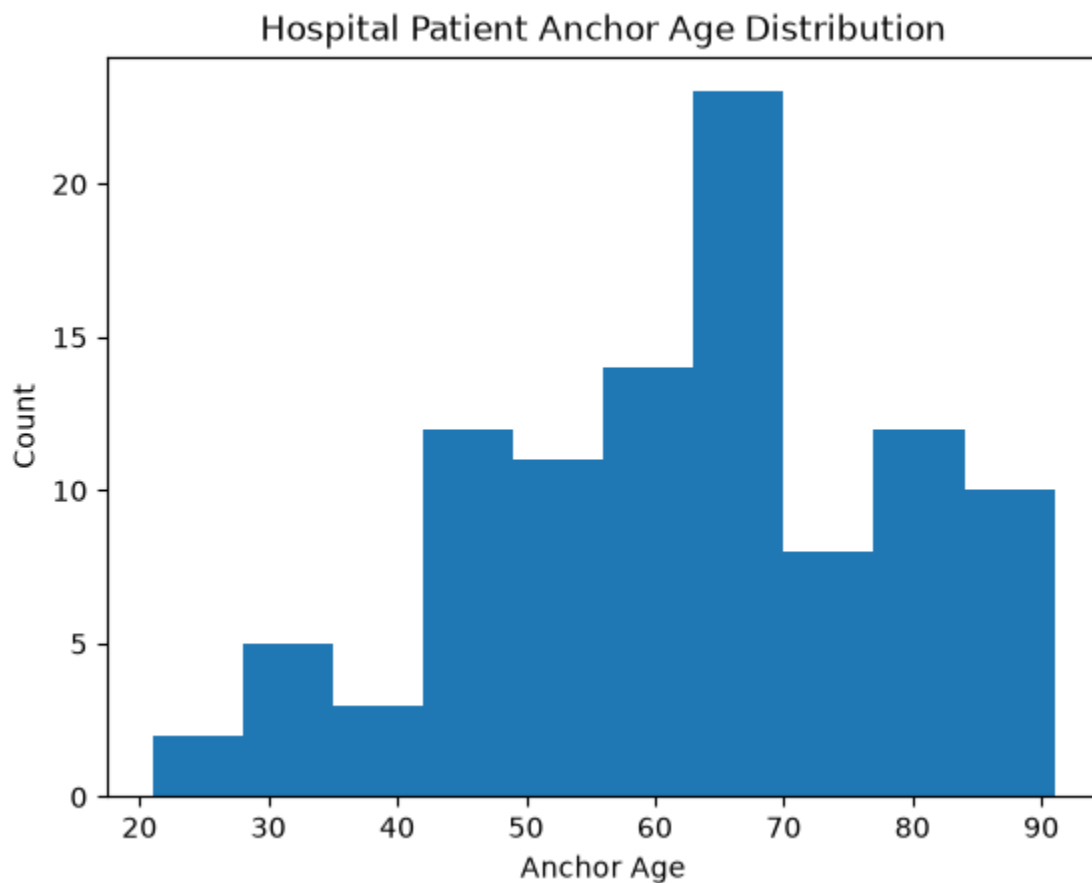
# num of each gender
pd.read_sql("SELECT gender, COUNT(*) AS n FROM hosp_patients GROUP BY gender", conn)

# duration of stay
durations = pd.read_sql("""
SELECT admission_type, (julianday(disctime) - julianday(admittime)) AS stay_du
FROM hosp_admissions
""", conn)

# plot duration of stay
plt.hist(durations["stay_duration"], bins=20)
plt.title("Duration of Hospital Patient Stays")
plt.xlabel("Days")
```



```
plt.ylabel("Count")  
plt.show()
```



Most admissions were emergency ward emergencies, with the average stay duration of emergency ward admissions being around 7 days.

```
In [8]: # dist of admission types
admissions = pd.read_sql("SELECT admission_type, COUNT(*) AS n FROM hosp_admissions
display(admissions)

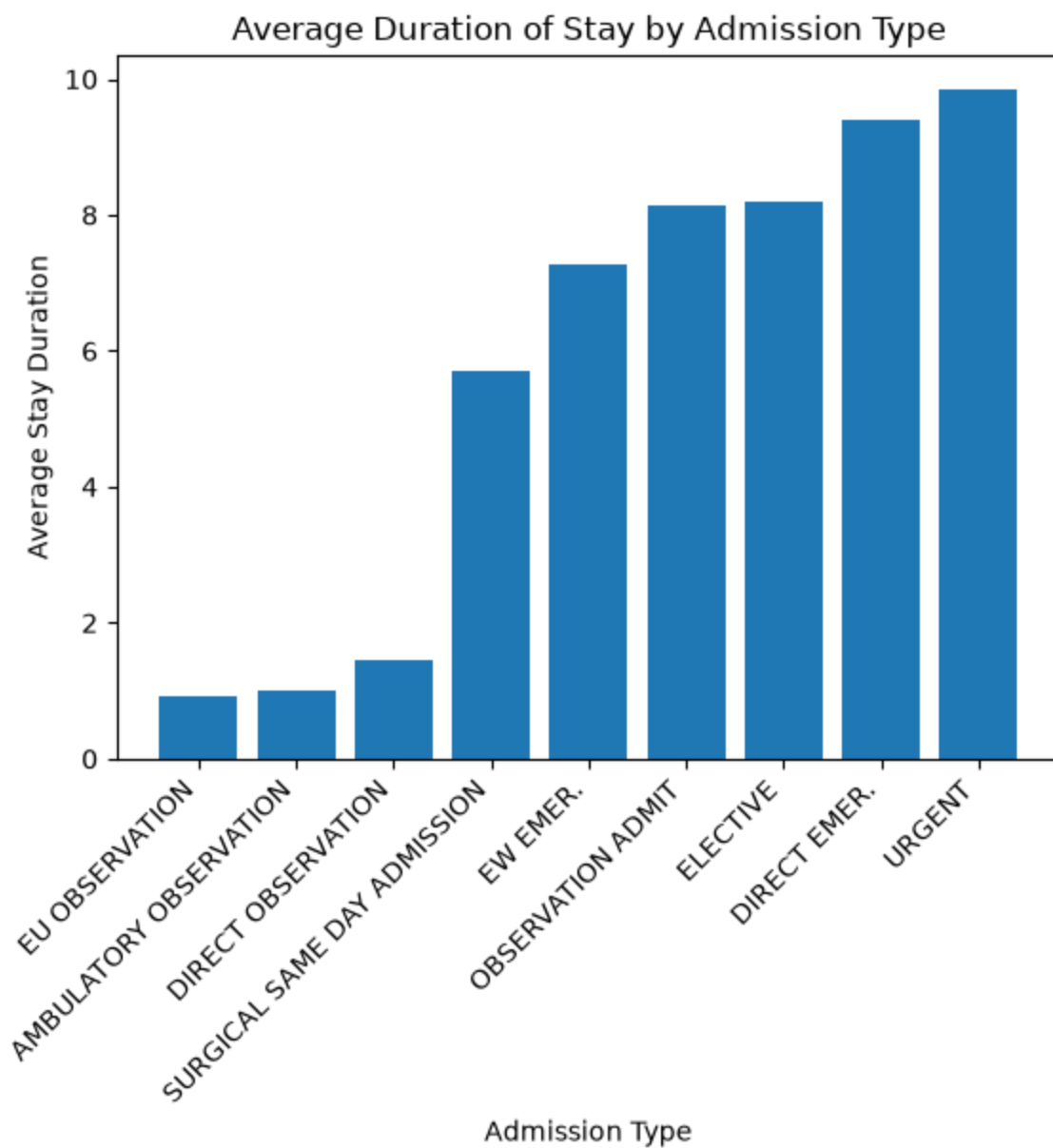
# remove table for tests
conn.execute("DROP TABLE IF EXISTS dur_by_type")

# compare avg stay times for each type of admission
conn.execute("""CREATE TABLE IF NOT EXISTS dur_by_type AS
                SELECT admission_type, AVG(stay_duration) as avg
                FROM (SELECT admission_type, (julianday(disctime) - julianday(admitti
                GROUP BY admission_type
                ORDER BY avg
                """)

duration_by_type = pd.read_sql("SELECT * FROM dur_by_type", conn)

plt.bar(duration_by_type["admission_type"], duration_by_type["avg"])
plt.title("Average Duration of Stay by Admission Type")
plt.xlabel("Admission Type")
plt.ylabel("Average Stay Duration")
plt.xticks(rotation=45, ha="right")
plt.show()
```

	admission_type	n
0	AMBULATORY OBSERVATION	5
1	DIRECT EMER.	15
2	DIRECT OBSERVATION	7
3	ELECTIVE	13
4	EU OBSERVATION	30
5	EW EMER.	104
6	OBSERVATION ADMIT	45
7	SURGICAL SAME DAY ADMISSION	18
8	URGENT	38



```
In [9]: readmissions = pd.read_sql("""
        WITH admit_pairs AS (
            SELECT
                a1.subject_id,
                a1.hadm_id,
                a1.dischtime,
                MIN(a2.admittime) as next_admittime
            FROM hosp_admissions a1
            LEFT JOIN hosp_admissions a2 ON
                a1.subject_id = a2.subject_id
                AND a2.admittime > a1.dischtime
            GROUP BY a1.subject_id, a1.hadm_id, a1.dischtime
        )
        SELECT *,
        CASE
            WHEN julianday(next_admittime) - julianday(dischtime) <= 30
            THEN 1 ELSE 0
```

```

        END AS readmitted_30_day
    FROM admit_pairs
""", conn)

display(readmissions.head(5))

```

	subject_id	hadm_id	disctime	next_admittime	readmitted_30_day
0	10000032	22595853	2180-05-07 17:15:00	2180-06-26 18:27:00	0
1	10000032	22841357	2180-06-27 18:49:00	2180-07-23 12:35:00	1
2	10000032	25742920	2180-08-07 17:50:00	NaN	0
3	10000032	29079034	2180-07-25 17:55:00	2180-08-05 23:44:00	1
4	10001217	24597018	2157-11-25 18:00:00	2157-12-18 16:58:00	1

```

In [10]: # select features for model
features_df = pd.read_sql("""
    SELECT a.hadm_id, p.gender, p.anchor_age, a.admission_type, a.race,
        julianday(a.disctime)-julianday(a.admittime) as los_days,
        CASE WHEN i.stay_id IS NOT NULL THEN 1 ELSE 0 END AS had_icu_stay,
        COALESCE(JULIANDAY(i.outtime) - JULIANDAY(i.intime), 0) AS icu_los_days
    FROM hosp_admissions a
    JOIN hosp_patients p ON a.subject_id = p.subject_id
    LEFT JOIN icu_icustays i ON a.hadm_id = i.hadm_id
""", conn)

# final df
final_df = readmissions.merge(features_df, on='hadm_id', how='left')

```

```

In [11]: from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# check class counts
print(final_df['readmitted_30_day'].value_counts())
print(final_df['readmitted_30_day'].value_counts(normalize=True))

# encode categorical variables
for col in ['gender', 'admission_type', 'race']:
    final_df[col] = LabelEncoder().fit_transform(final_df[col].astype(str))

X = final_df[['anchor_age', 'gender', 'admission_type', 'race',
    'los_days', 'had_icu_stay', 'icu_los_days']]
y = final_df['readmitted_30_day']

# train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

```

```
readmitted_30_day
0    232
1     55
Name: count, dtype: int64
readmitted_30_day
0    0.808362
1    0.191638
Name: proportion, dtype: float64
```

```
In [12]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report
import numpy as np

# fit model
model = LogisticRegression(class_weight='balanced')
model.fit(X_train, y_train)

# evaluate
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

# roc auc scores
print(f"AUC: {roc_auc_score(y_test, y_prob):.3f}")
print(classification_report(y_test, y_pred))

# coefficient table
coef_df = pd.DataFrame({
    'feature': X.columns,
    'coefficient': model.coef_[0],
    'odds_ratio': np.exp(model.coef_[0])
}).sort_values('odds_ratio', ascending=False)

print(coef_df)
```

AUC: 0.462

	precision	recall	f1-score	support
0	0.81	0.65	0.72	46
1	0.24	0.42	0.30	12
accuracy			0.60	58
macro avg	0.52	0.53	0.51	58
weighted avg	0.69	0.60	0.64	58

	feature	coefficient	odds_ratio
5	had_icu_stay	0.191014	1.210476
2	admission_type	0.076370	1.079362
4	los_days	0.038885	1.039651
3	race	0.036136	1.036797
0	anchor_age	-0.022670	0.977585
6	icu_los_days	-0.117418	0.889214
1	gender	-0.956337	0.384298